CS 170 Extra-Credit #6

The Game of Nim

The game of Nim is a well-known game with a number of variants. For this project, you'll build the following variant, which has an interesting winning strategy. Two players alternately take marbles from a pile. In each move, a player chooses how many marbles to take. The player must take at least one, but at most half of the marbles. Then, the other player takes a turn. The player who takes the last marble loses.

Here's one sample run to give you an example of how the program should act:

```
Welcome to the game of Nim. Pile Size: 67 marbles

Your move. How many would you like to take? 5

Current pile size: 62 marbles. I'll take: 21
Current pile size: 41 marbles. How many would you like to take? 11
Current pile size: 30 marbles. I'll take: 2
Current pile size: 28 marbles. How many would you like to take? 8
Current pile size: 20 marbles. I'll take: 9
Current pile size: 11 marbles. How many would you like to take? 8
Illegal Move. Maximum you may take is 5 marbles. Try Again. 3
Current pile size: 8 marbles. I'll take: 4
Current pile size: 4 marbles. How many would you like to take? 1
Current pile size: 3 marbles. I'll take: 1
Current pile size: 2 marbles. How many would you like to take? 1

Current pile size: 1 marbles. I'll take: 1

Current pile size: 0 marbles
YOU BEAT ME!!

Thank you for playing
```

To start this off, select **Project Open** in **DrJava** and navigate to the **NIM-Project** folder in the Chapter12 code folder. This folder contains a complete unzipped copy of the ACM classes so you can create an "executable" Jar file from your project.

The main file, **Nim.java** is an ACM console-mode program, (so you don't need to get bogged down with graphics or the user interface; instead, you'll concentrate on designing the classes and the logic), in which the computer plays against a human opponent.

When you implement your game, use the classes **`Nim`**, **`Pile`**, **`Player`** and **`Game`**. The **`Nim`** class should contain a **`run()`** method that looks **exactly** like this. Do not change this method, **except to add a line before the loop, that prints your name in the form: Author: Last, First**. Your program MUST have this line to get credit for this assignment. You may also change the colors, fonts, etc, in the marked location.

```
public void run()
{
    // Set up the console, colors, etc. (Optional)
    do
    {
        println("Welcome to the game of Nim!");

        // Create a Game object, passing it the IOConsole
        Game nim = new Game(cons);
        nim.play();

    } while (readLine("\nPlay again (y/n)? ").startsWith("y"));

    println("Thanks for playing!!!");
}
```

Notice that the **`run()`** method simply creates a **`Game`** object passing it an **`IOConsole`** object. Your **`Game`** constructor should save this in an instance variable and then use it to print on the screen with **`cons.println()`**. That way, each class is sharing the same screen. The **`Game`** object will create the two **`Player`** objects as well as a **`Pile`** object. A player can be either: a)stupid, b) smart or c) human. Human **`Player`** objects will prompt for input, while stupid and smart players will simply play according the appropriate strategy. (Your **`Player`** constructor will probably need a **`IOConsole`** parameter as well, so it can get input and print messages.)

When you call the **`Game`**'s **`play()`** method, the **`Game`** object should generate a random integer between 10 and 100 to denote the initial size of the pile and then create its **`Pile`** object. Then, generate a random integer between 0 and 1 to decide whether the human or the computer takes the first turn. Also, when you create your computer player, generate a random number between 0 and 1 to decide whether the computer plays *smart* or *stupid*.

In stupid mode, the computer simply takes a random legal value (between 1 and n/2) from the pile whenever it has a turn. In smart mode the computer takes off enough marbles to make the size of the pile a power of two minus 1 (that is, 3, 7, 15, 31, or 63). That is always a legal move, except if the size of the pile is currently one less than a power of 2. In that case, the computer makes a random legal move. *Note that the computer cannot be beaten in smart mode when it has the first move, unless the pile size happens to be 15, 31, or 63. Of course, a human player who has the first turn and knows the winning strategy can win against the computer.*

When you have the game working as you like, use **Project->Create Jar File from Project** to package everything in a single "executable" named **`NIM.jar`**. Include the Source files in your **`JAR`** file and submit it following your instructor's directions.